- **EXERCISE PART 1: UNDERSTANDING PROJECT STRUCTURE**

1. Exploring the Codebase

- Navigated through the JavaScript Task Manager project folder.
- Reviewed package.json to understand the project dependencies and configuration.
- Identified the main files:
    - cli.js
    - storage.js
    - task.js
    - task_list.js

2. Form Initial Understanding

Codebase Organisation

- The project uses a modular structure with different responsibilities separated into different files.
- This makes the code easier to read, maintain, and update.

Technologies and Frameworks

- **JavaScript (Node.js):** Main language and runtime.
- **Commander:** Handles command-line inputs and commands.
- **UUID:** Generates unique IDs for tasks.
- **Jest:** Used for automated testing.

Main Components

- **cli.js** – Manages user interaction and command execution.
- **storage.js** – Saves and loads task data.
- **task.js & task_list.js** – Contain the core task management logic, including creating, updating, and deleting tasks.

3. Applying the Project Structure Prompt

Prompt Used

- *"Understanding the project structure and technology stack"*

Initial Understanding

- Believed that the project was a command-line to-do list application.
- Question asked: *"Does storage.js save data to a database or local file?"*

AI Findings

- Confirmed that it is a modular CLI application.
- Clarified that storage.js most likely stores tasks in a local JSON file using Node.js file system functionality rather than a database.

4. Findings

Misconceptions

- Initially expected a requirements.txt file.
- Learned that Node.js projects use package.json instead of Python's dependency file.

Entry Point & Architecture

- **Entry Point:** cli.js
- **Architecture:** Separation of Concerns
    - User Input → Business Logic → Data Storage

Key Responsibilities

- **cli.js** – Parses terminal commands and triggers actions.
- **task_list.js** – Manages the collection of tasks.
- **task.js** – Defines individual tasks, including ID, description, and status.
- **storage.js** – Persists task data locally so it remains available between sessions.

- **EXERCISE PART 2: FINDING FEATURE IMPLEMENTATION (CSV EXPORT)**

1. Initial Search

- Reviewed cli.js to understand how commands are registered.
- Examined storage.js to see how data is saved and loaded.
- Checked task_list.js to understand how tasks are stored and retrieved.

2. Form Hypothesis

- Add a new export command in cli.js.
- Retrieve the current task list from task_list.js.
- Handle CSV formatting and file creation in storage.js to keep responsibilities separated.

3. Apply the Feature Location Prompt

Prompt Used

- *"I need to add a feature that exports the current to-do list to a CSV file. Based on the current architecture, which files should I modify and where should the core logic go?"*

AI Findings

- Confirmed that cli.js should handle the new command.
- Recommended placing CSV export logic in storage.js.
- Suggested using Node.js file system functions and simple string formatting instead of an external CSV library.

- **4. Document Findings**

  Misconceptions
  - Initially thought CSV conversion belonged in task_list.js.
  - Realised file formatting should be handled by storage.js to maintain separation of concerns.

  Entry Point
  - cli.js is the main entry point for the export feature.

  Implementation Plan
  - **Step 1:** Add an export command in cli.js.
  - **Step 2:** Retrieve all tasks from task_list.js.
  - **Step 3:** Implement an exportToCSV() function in storage.js to format the data and save it as a .csv file.

- **EXERCISE PART 3: UNDERSTANDING THE DOMAIN MODEL**

  1. Extract the Domain Model
  - Reviewed task.js and task_list.js to identify the main entities.
  - Examined task properties such as ID, description, and status.
  - Analysed methods to understand how tasks are created and managed.

  2. Form Initial Understanding

  Core Entities
  - **Task** – Represents a single to-do item.
  - **TaskList** – Manages a collection of tasks.

  Task Properties
  - Unique ID (UUID)
  - Description
  - Completion status

  Relationship
  - One TaskList contains multiple Task objects.

  3. Apply the Domain Model Prompt

  Prompt Used
  - *"Explain the domain model of this application based on the codebase. What are the core business entities, their properties, and how do they interact?"*

AI Findings

- Confirmed that Task and TaskList are the main domain entities.
- Verified that there are no additional entities such as User or Category in the current application.

4. Document Findings

Misconceptions

- Initially considered storage.js part of the domain model.
- Learned that it is an infrastructure component responsible for data persistence rather than business logic.

Key Relationships

- A TaskList contains many Task objects.
- Each Task is managed through the TaskList.

Business Rules

- Every task has a unique UUID.
- Each task includes a description and a completion status.
- Tasks can be added, updated, completed, or removed through the TaskList.

- **EXERCISE PART 4: PRACTICAL APPLICATION**

1. Scenario

- Implement a rule that automatically marks tasks as abandoned if they are overdue by more than 7 days, unless they are marked as high priority.

2. Planning

Files to Modify

- **task.js** – Review task properties such as status, priority, and due date.
- **task_list.js** – Add logic to evaluate overdue tasks and update their status.
- **storage.js** – Ensure updated task statuses are saved correctly if needed.

Outline of Changes

- Compare each task's due date with the current date.
- Check whether the task is more than 7 days overdue.
- Skip tasks marked as high priority.
- Update qualifying tasks to "abandoned" and save the changes.

Questions for the Team

- How is high priority represented in the application?
- Is there an existing method for updating task statuses?
- Should abandoned tasks still be editable or recoverable?

3. Reflection

AI Assistance

- AI helped identify the likely files to modify and suggested where the new business logic should be implemented while keeping the code organised.

Unclear Aspects

- It is unclear whether the project already supports priority levels and due dates.
- The process for automatically running the status update needs confirmation.

Next Steps

- Review the data model for priority and due date fields.
- Locate any existing status update methods.
- Implement the rule and create tests to verify the new behaviour.

- **FINAL REFLECTION**

  Personal Reflection

  Most Helpful Prompt

  - The Feature Location Prompt was the most useful because it helped identify the correct files to modify and maintain a clean architecture.

  What I Would Do Differently

  - Next time, I would map out the domain model and key components before exploring the implementation details.

  Additional Tools and Resources

  - An IDE debugger for stepping through code.
  - UML or architecture diagrams to visualise relationships.
  - AI prompts and official documentation to better understand unfamiliar code.